University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Final Project | Spring 2026

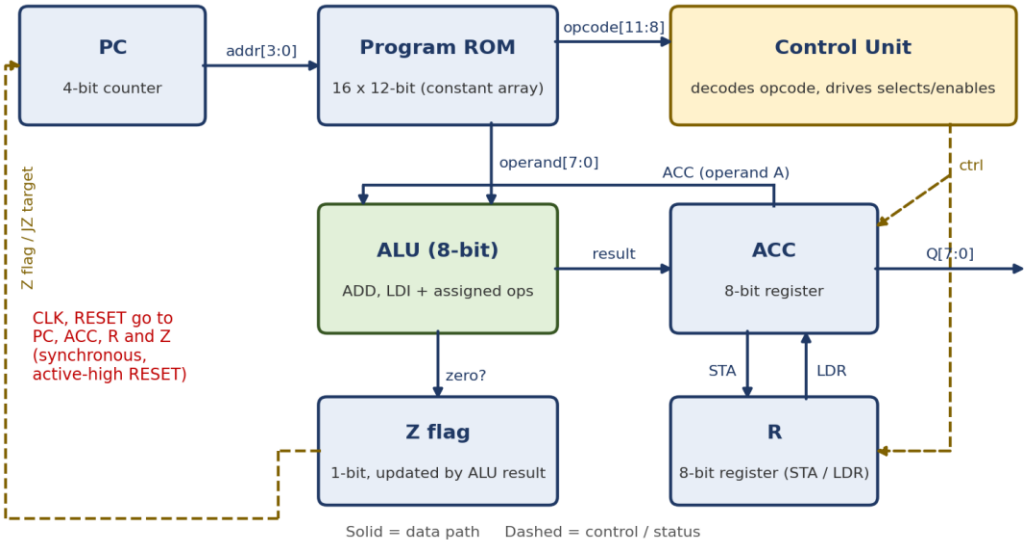
Total Marks	50 (Viva → 30, Report & VrC → 10, Presentation → 10)		
Student Name	Nazmul Mridha	**ID**	22201161

Final Project: Mini 8-bit Accumulator CPU

Objective. Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the “Instruction Set Assignment” page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.



Block	Description
PC	4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken.
Program ROM	16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP).
Control Unit	Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC.
ALU	8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required.
ACC, R, Z	ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00.

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	**LDI, ADD**	**SUB, AND, OR**	**LDI, ADD, SUB, AND, OR**
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
5	**Set 5**	LDI, ADD	OR, NOT, SHR	LDI, ADD, OR, NOT, SHR
6	**Set 6**	LDI, ADD	SUB, JZ, SHL	LDI, ADD, SUB, JZ, SHL
7	**Set 7**	LDI, ADD	AND, XOR, SHR	LDI, ADD, AND, XOR, SHR
8	**Set 8**	LDI, ADD	STA, LDR, SUB	LDI, ADD, STA, LDR, SUB
9	**Set 9**	LDI, ADD	NOT, SHL, JZ	LDI, ADD, NOT, SHL, JZ

Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

Set 0: LDI, ADD, AND, JZ, NOT

ROM contents: 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xA5	0xA5	0	10100101	1
2	1	AND 0x5A	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	NOT	0xFF	0	11111111	6
5	6	AND 0x0F	0x0F	0	00001111	7
6	7	JZ 0x00 (not taken)	0x0F	0	00001111	8
7	8	ADD 0x01	0x10	0	00010000	9

Set 1: LDI, ADD, SUB, AND, OR

ROM contents: 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x3C	0x3C	0	00111100	1
2	1	ADD 0x0F	0x4B	0	01001011	2
3	2	SUB 0x0B	0x40	0	01000000	3
4	3	AND 0xF0	0x40	0	01000000	4
5	4	OR 0x0A	0x4A	0	01001010	5
6	5	SUB 0x4A	0x00	1	00000000	6
7	6	SUB 0x01	0xFF	0	11111111	7

Set 2: LDI, ADD, XOR, NOT, SHL

ROM contents: 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x55	0x55	0	01010101	1
2	1	XOR 0xFF	0xAA	0	10101010	2
3	2	NOT	0x55	0	01010101	3
4	3	SHL	0xAA	0	10101010	4
5	4	ADD 0x56	0x00	1	00000000	5
6	5	XOR 0x81	0x81	0	10000001	6
7	6	SHL	0x02	0	00000010	7

Set 3: LDI, ADD, SHR, SUB, XOR

ROM contents: 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x80	0x80	0	10000000	1
2	1	SHR	0x40	0	01000000	2
3	2	SUB 0x10	0x30	0	00110000	3
4	3	ADD 0x0F	0x3F	0	00111111	4
5	4	XOR 0x3F	0x00	1	00000000	5
6	5	ADD 0x07	0x07	0	00000111	6
7	6	SHR	0x03	0	00000011	7

Set 4: LDI, ADD, STA, LDR, AND

ROM contents: 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x1E	0x1E	0	0x00	00011110	1
2	1	STA	0x1E	0	0x1E	00011110	2
3	2	ADD 0x22	0x40	0	0x1E	01000000	3
4	3	AND 0x60	0x40	0	0x1E	01000000	4
5	4	LDR	0x1E	0	0x1E	00011110	5
6	5	ADD 0x01	0x1F	0	0x1E	00011111	6
7	6	AND 0x0F	0x0F	0	0x1E	00001111	7
8	7	STA	0x0F	0	0x0F	00001111	8
9	8	LDI 0x00	0x00	1	0x0F	00000000	9
10	9	LDR	0x0F	0	0x0F	00001111	10

Set 5: LDI, ADD, OR, NOT, SHR

ROM contents: 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x0F	0x0F	0	00001111	1
2	1	OR 0xA0	0xAF	0	10101111	2
3	2	NOT	0x50	0	01010000	3
4	3	SHR	0x28	0	00101000	4
5	4	ADD 0xD8	0x00	1	00000000	5
6	5	OR 0x81	0x81	0	10000001	6
7	6	SHR	0x40	0	01000000	7

Set 6: LDI, ADD, SUB, JZ, SHL

ROM contents: 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x02	0x02	0	00000010	1
2	1	SUB 0x01	0x01	0	00000001	2
3	2	JZ 0x07 (not taken)	0x01	0	00000001	3
4	3	SUB 0x01	0x00	1	00000000	4
5	4	JZ 0x07 (taken)	0x00	1	00000000	7
6	7	LDI 0x21	0x21	0	00100001	8
7	8	SHL	0x42	0	01000010	9
8	9	ADD 0x01	0x43	0	01000011	10

Set 7: LDI, ADD, AND, XOR, SHR

ROM contents: 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xF3	0xF3	0	11110011	1
2	1	AND 0x3C	0x30	0	00110000	2
3	2	XOR 0x11	0x21	0	00100001	3
4	3	SHR	0x10	0	00010000	4
5	4	ADD 0x30	0x40	0	01000000	5
6	5	XOR 0x40	0x00	1	00000000	6

Set 8: LDI, ADD, STA, LDR, SUB

ROM contents: 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x64	0x64	0	0x00	01100100	1
2	1	STA	0x64	0	0x64	01100100	2
3	2	SUB 0x14	0x50	0	0x64	01010000	3
4	3	ADD 0x0A	0x5A	0	0x64	01011010	4
5	4	STA	0x5A	0	0x5A	01011010	5
6	5	SUB 0x5A	0x00	1	0x5A	00000000	6
7	6	LDR	0x5A	0	0x5A	01011010	7
8	7	SUB 0x60	0xFA	0	0x5A	11111010	8

Set 9: LDI, ADD, NOT, SHL, JZ

ROM contents: 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xFF	0xFF	0	11111111	1
2	1	NOT	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	ADD 0x11	0x11	0	00010001	6
5	6	SHL	0x22	0	00100010	7
6	7	JZ 0x00 (not taken)	0x22	0	00100010	8
7	8	SHL	0x44	0	01000100	9

Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

Final Project Report

Project Title: Mini 8-bit Accumulator CPU (VHDL, Set 1)

Submitted by:

Name: Nazmul Mridha
Student ID: 22201161
Section: C2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 29/10/2026

1. Student Information

Field	Details
Student Name	Nazmul Mridha
Student ID	**22201161**
Last digit of ID / Assigned Set	**1 → Set 1**
Mandatory instructions	LDI, ADD
Assigned instructions	**SUB, AND, OR**
Complete instruction set of the CPU	LDI, ADD, SUB, AND, OR
Language / Tools	VHDL; Xilinx ISE (XST synthesis, ISim simulator)

Instruction set of the CPU (Set 1):

Mnemonic	Opcode	Operation	Z flag	Type
LDI imm	0001	ACC ← imm	Updated	Mandatory
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Updated	Mandatory
SUB imm	**0011**	**ACC ← ACC − imm (modulo 256)**	**Updated**	**Assigned**
AND imm	**0100**	**ACC ← ACC AND imm**	**Updated**	**Assigned**
OR imm	**0101**	**ACC ← ACC OR imm**	**Updated**	**Assigned**

Any other opcode (including 0000) is treated as a NOP: ACC and Z are unchanged and the program counter advances by one. Set 1 contains neither STA nor LDR, so the holding register R is not required, and it contains no JZ, so the program counter always increments by one.

2. Objective

The objective of this project is to design and implement a small single-cycle 8-bit accumulator CPU in VHDL that executes the five instructions of Set 1 (LDI, ADD, SUB, AND and OR). The CPU is composed of modular blocks: a 4-bit program counter, a 16 × 12-bit program ROM, a control unit, an 8-bit ALU and the ACC and Z registers. All sequential blocks are clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET. The complete CPU is verified with a self-checking testbench that executes the Set 1 program given in Appendix B and compares ACC, Z and PC with the expected values after every clock edge.

3. System Architecture

3.1 Instruction format

Each instruction is 12 bits wide: instr[11:8] is the 4-bit opcode and instr[7:0] is the 8-bit operand (immediate). Every instruction is executed in a single clock cycle: the program counter addresses the ROM, the control unit decodes the opcode, the ALU computes the result, and the result is written to ACC at the rising edge of the clock while the program counter advances by one.

Opcode instr[11:8]	**Operand instr[7:0]**

Example: LDI 0x3C = 0001 00111100 = 0x13C.

3.2 Functional blocks

Block	Description	VHDL file
Program Counter (PC)	4-bit counter holding the address of the current instruction. It becomes PC + 1 after every instruction (Set 1 contains no JZ) and is cleared to 0 by RESET.	pc.vhd
Program ROM	16 × 12-bit constant array, read combinationally using PC (no clock). Unused locations contain 0x000 (NOP).	rom.vhd
Control Unit	Decodes opcode instr[11:8] and generates the 3-bit ALU function select alu_sel and the write enables acc_we and z_we.	control.vhd
ALU	8-bit combinational unit operating on ACC and the operand: LDI (passes the operand), ADD, SUB, AND and OR.	alu.vhd
ACC, Z	ACC: 8-bit accumulator and CPU output Q[7:0]. Z: 1-bit flag that is 1 when the value written to ACC is 0x00. Reset values: ACC = 0, Z = 1.	acc_z.vhd
CPU top level	Connects all blocks. Inputs: CLK, RESET. Outputs: Q[7:0], Z, PC[3:0].	cpu.vhd
Testbench	Generates the clock and RESET, runs the Appendix B program and checks the results.	cpu_tb.vhd

3.3 Design hierarchy

```
cpu_tb                          (cpu_tb.vhd  - testbench)
└── DUT   : cpu                 (cpu.vhd     - top level)
    ├── u_pc  : pc              (pc.vhd)
    ├── u_rom : rom             (rom.vhd)
    ├── u_ctl : control         (control.vhd)
    ├── u_alu : alu             (alu.vhd)
    └── u_reg : acc_z           (acc_z.vhd)
```

The ALU function select used between the control unit and the ALU is encoded as follows:

alu_sel	Instruction	ALU result
000	LDI	operand
001	ADD	ACC + operand (8-bit, carry discarded)
010	**SUB**	**ACC − operand (modulo 256)**
011	**AND**	**ACC and operand**
100	**OR**	**ACC or operand**

4. Module-Level Design

The CPU was built from five small VHDL entities. For every module the description, the source code, the RTL schematics (top view and expanded view generated by Xilinx ISE), the testbench and its simulation waveform are given below.

4.1 Program Counter (pc.vhd)

A 4-bit counter that holds the address of the current instruction. On every rising edge of clk it increments by one; when reset is high it is cleared to 0 (synchronous, active-high reset). Because Set 1 has no JZ instruction, no jump-load path is needed. The counter wraps from 15 back to 0 naturally.

Source code (pc.vhd):

```
-- Step 1: Program Counter (4-bit). Sync active-high reset. PC <= PC + 1 each clock.
-- Set 1 has no JZ, so no load/jump input is needed.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pc is
  port ( clk   : in  std_logic;
         reset : in  std_logic;
         pc_out: out std_logic_vector(3 downto 0) );
end pc;

architecture rtl of pc is
  signal cnt : unsigned(3 downto 0) := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if reset = '1' then cnt <= (others => '0');
      else                cnt <= cnt + 1;
      end if;
    end if;
  end process;
  pc_out <= std_logic_vector(cnt);
end rtl;
```

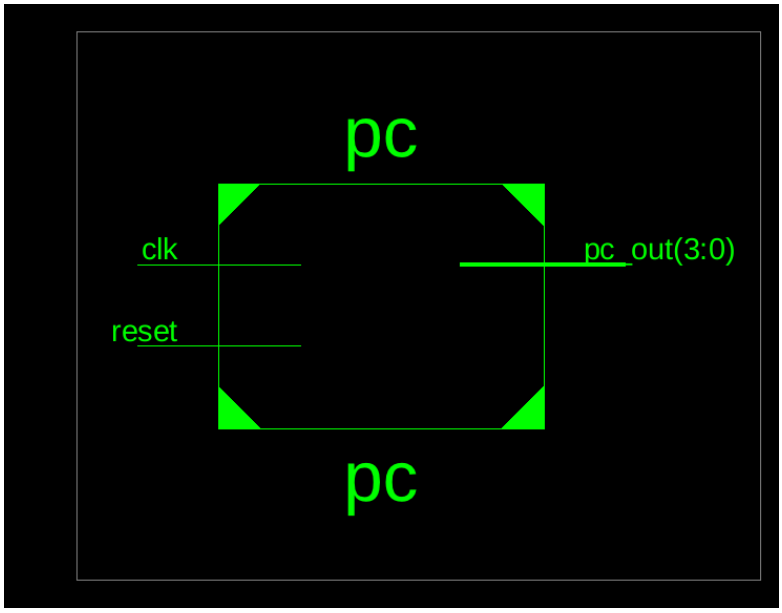


Figure 1: Top view of RTL schematic of the program counter (pc)

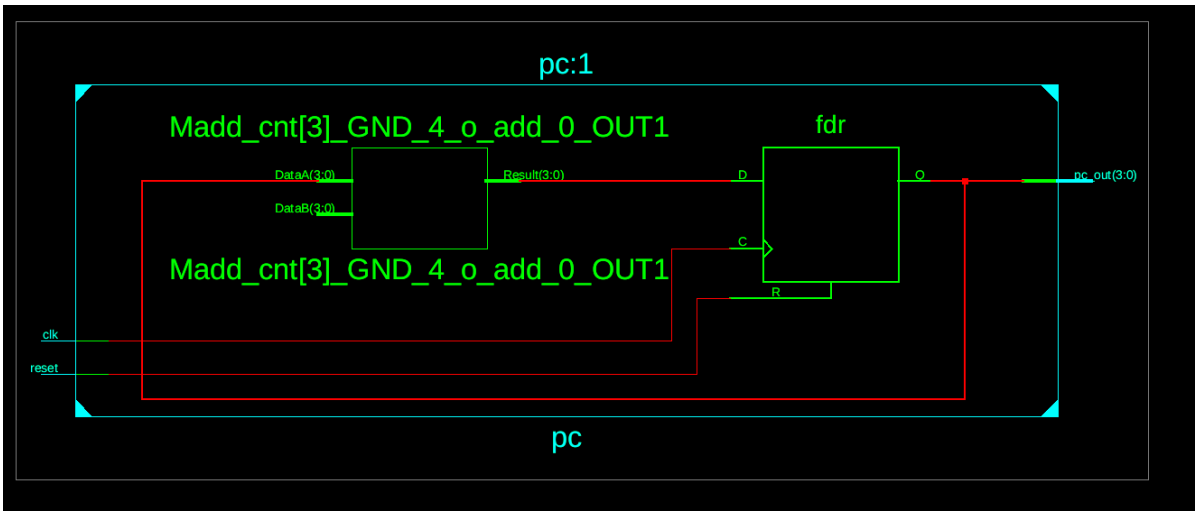


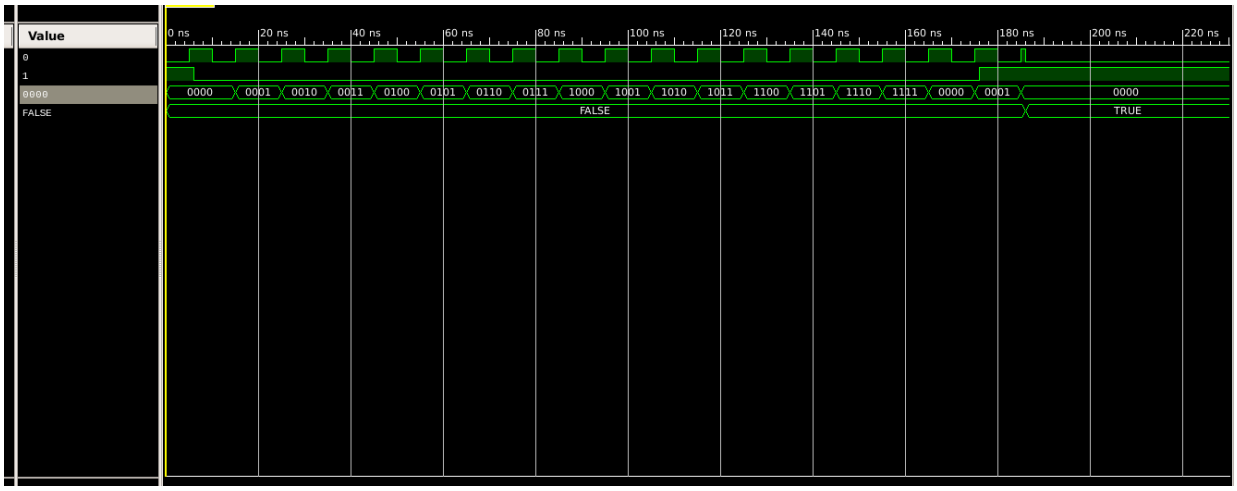
Figure 2: Expanded view of RTL schematic of the program counter (pc)

The expanded schematic shows a 4-bit adder (PC + 1) feeding an FDR flip-flop, which is a D flip-flop with synchronous reset. The flip-flop output is fed back to the adder and is also the output pc_out.

Source code (pc_tb.vhd):

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.ALL;
entity pc_tb is end pc_tb;
architecture sim of pc_tb is
  signal clk, reset : std_logic := '0';
  signal pc_out : std_logic_vector(3 downto 0);
  signal done : boolean := false;
begin
  dut : entity work.pc port map (clk, reset, pc_out);
  clk <= not clk after 5 ns when not done else '0';
  process
    variable errors : integer := 0;
  begin
    reset <= '1'; wait until rising_edge(clk); wait for 1 ns;
    assert pc_out = x"0" report "Reset failed" severity error;
    reset <= '0';
    for i in 1 to 17 loop                        -- counts up and wraps 15 -> 0
      wait until rising_edge(clk); wait for 1 ns;
      if pc_out /= std_logic_vector(to_unsigned(i mod 16, 4)) then
        report "PC mismatch at step " & integer'image(i) severity error; errors := errors + 1;
      end if;
    end loop;
    reset <= '1'; wait until rising_edge(clk); wait for 1 ns;   -- reset mid-run
    if pc_out /= x"0" then report "Mid-run reset failed" severity error; errors := errors + 1; end if;
    if errors = 0 then report "PC_TB: ALL TESTS PASSED"; end if;
    done <= true; wait;
  end process;
end sim;
```

Waveform:



reset is high for the first rising edge, so pc_out = 0000. After reset is released the counter increments by one on every rising edge (0001, 0010, …, 1111) and wraps from 1111 to 0000. When reset is asserted again (at about 175 ns) the counter returns to 0000 at the next rising edge, and done becomes TRUE which stops the clock.

4.2 Program ROM (rom.vhd)

A 16 × 12-bit read-only memory written as a constant array and read combinationally using the 4-bit address (no clock). Locations 0 to 6 hold the Set 1 program of Appendix B; locations 7 to 15 contain 0x000 (NOP). Each word is the 4-bit opcode followed by the 8-bit operand.

Address	Instruction	Machine code	Binary (opcode & operand)
0	LDI 0x3C	0x13C	0001 & 00111100
1	ADD 0x0F	0x20F	0010 & 00001111
2	**SUB 0x0B**	**0x30B**	**0011 & 00001011**
3	**AND 0xF0**	**0x4F0**	**0100 & 11110000**
4	**OR 0x0A**	**0x50A**	**0101 & 00001010**
5	SUB 0x4A	0x34A	0011 & 01001010
6	SUB 0x01	0x301	0011 & 00000001
7 – 15	NOP	0x000	0000 & 00000000

Source code (rom.vhd):

```
-- Step 2: Program ROM, 16 x 12 bits, combinational read. Appendix B, Set 1.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity rom is
  port ( addr  : in  std_logic_vector(3 downto 0);
         instr : out std_logic_vector(11 downto 0) );
end rom;

architecture rtl of rom is
  type rom_t is array (0 to 15) of std_logic_vector(11 downto 0);
  constant PROG : rom_t := (
    0 => x"13C",   -- LDI 0x3C
    1 => x"20F",   -- ADD 0x0F
    2 => x"30B",   -- SUB 0x0B
    3 => x"4F0",   -- AND 0xF0
    4 => x"50A",   -- OR  0x0A
    5 => x"34A",   -- SUB 0x4A
    6 => x"301",   -- SUB 0x01
    others => x"000");  -- NOP
begin
  instr <= PROG(to_integer(unsigned(addr)));
end rtl;
```

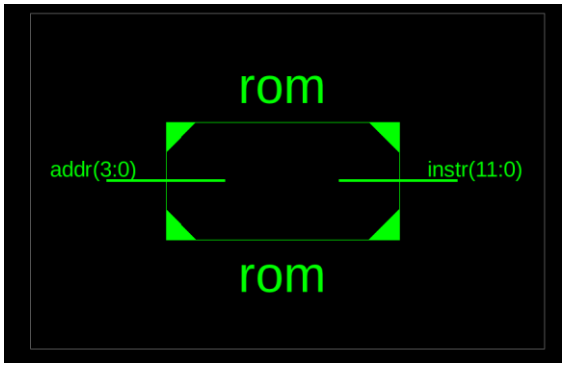


Figure 3: Top view of RTL schematic of the program ROM (rom)

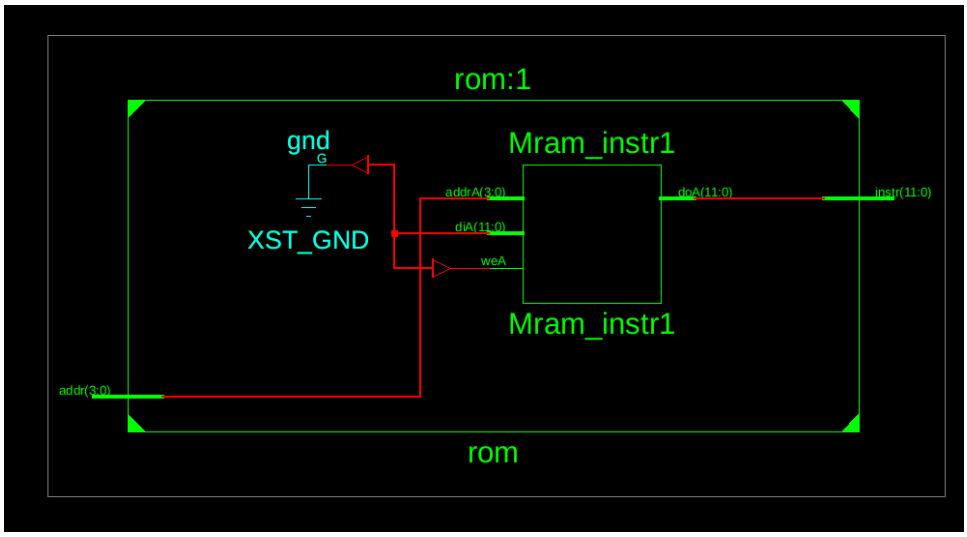


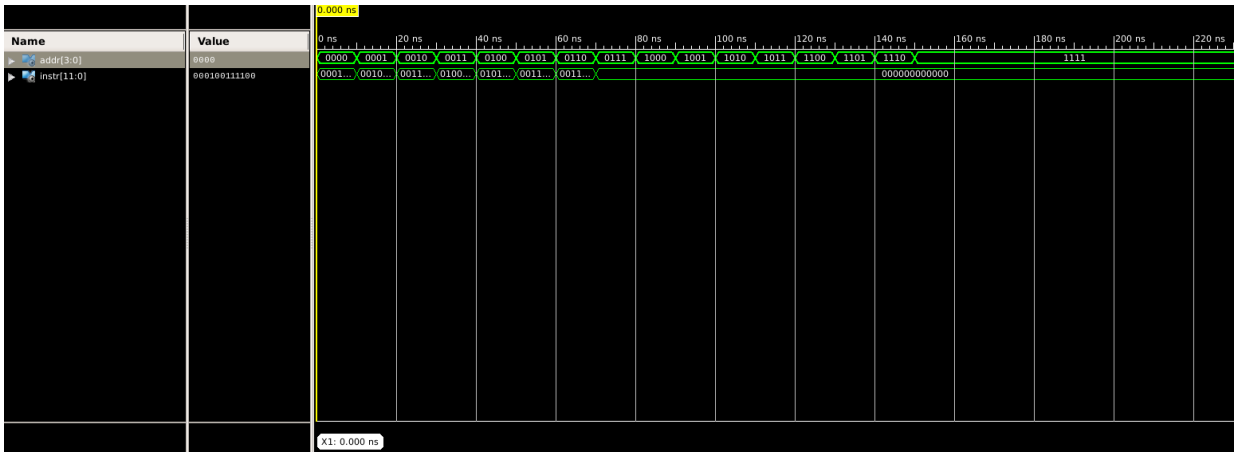
Figure 4: Expanded view of RTL schematic of the program ROM (rom)

The synthesis tool inferred the constant array as a memory block (Mram_instr1) with a 4-bit address input and a 12-bit data output. Its write-data and write-enable inputs are tied to ground, which confirms that the memory is read-only.

Source code (rom_tb.vhd):

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.ALL;
entity rom_tb is end rom_tb;
architecture sim of rom_tb is
  signal addr  : std_logic_vector(3 downto 0) := (others => '0');
  signal instr : std_logic_vector(11 downto 0);
  type exp_t is array (0 to 15) of std_logic_vector(11 downto 0);
  constant EXP : exp_t := (x"13C",x"20F",x"30B",x"4F0",x"50A",x"34A",x"301",
                           others => x"000");
begin
  dut : entity work.rom port map (addr, instr);
  process
    variable errors : integer := 0;
  begin
    for i in 0 to 15 loop
      addr <= std_logic_vector(to_unsigned(i, 4)); wait for 10 ns;
      if instr /= EXP(i) then
        report "ROM mismatch at address " & integer'image(i) severity error; errors := errors + 1;
      end if;
    end loop;
    if errors = 0 then report "ROM_TB: ALL TESTS PASSED"; end if;
    wait;
  end process;
end sim;
```

Waveform:



addr is stepped from 0000 to 1111 in 10 ns steps. instr shows 13C, 20F, 30B, 4F0, 50A, 34A and 301 for addresses 0 to 6 and 000 (NOP) for addresses 7 to 15, exactly as stored in the ROM.

4.3 Control Unit (control.vhd)

A combinational decoder that converts the 4-bit opcode into the control signals alu_sel, acc_we and z_we. All outputs default to NOP, therefore unused opcodes, including 0000, leave ACC and Z unchanged. In Set 1 every implemented instruction writes both ACC and Z, so acc_we and z_we are always equal.

Opcode	Instruction	alu_sel	acc_we	z_we
0001	LDI	000	1	1
0010	ADD	001	1	1
0011	**SUB**	**010**	**1**	**1**
0100	**AND**	**011**	**1**	**1**
0101	**OR**	**100**	**1**	**1**
others (incl. 0000)	NOP	000	0	0

Source code (control.vhd):

```
-- Step 3: Control Unit. Decodes opcode -> ALU select + write enables.
-- alu_sel: 000 pass imm (LDI) | 001 ADD | 010 SUB | 011 AND | 100 OR
-- Anything else (incl. 0000) is NOP: no enables asserted.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity control is
  port ( opcode  : in  std_logic_vector(3 downto 0);
         alu_sel : out std_logic_vector(2 downto 0);
         acc_we  : out std_logic;
         z_we    : out std_logic );
end control;

architecture rtl of control is
begin
  process(opcode)
  begin
    alu_sel <= "000"; acc_we <= '1'; z_we <= '1';
    case opcode is
      when "0001" => alu_sel <= "000";   -- LDI
      when "0010" => alu_sel <= "001";   -- ADD
      when "0011" => alu_sel <= "010";   -- SUB
      when "0100" => alu_sel <= "011";   -- AND
      when "0101" => alu_sel <= "100";   -- OR
      when others => acc_we <= '0'; z_we <= '0';  -- NOP
    end case;
  end process;
end rtl;
```

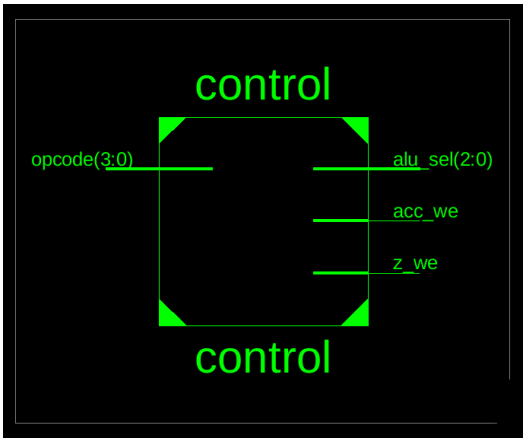


Figure 5: Top view of RTL schematic of the control unit (control)

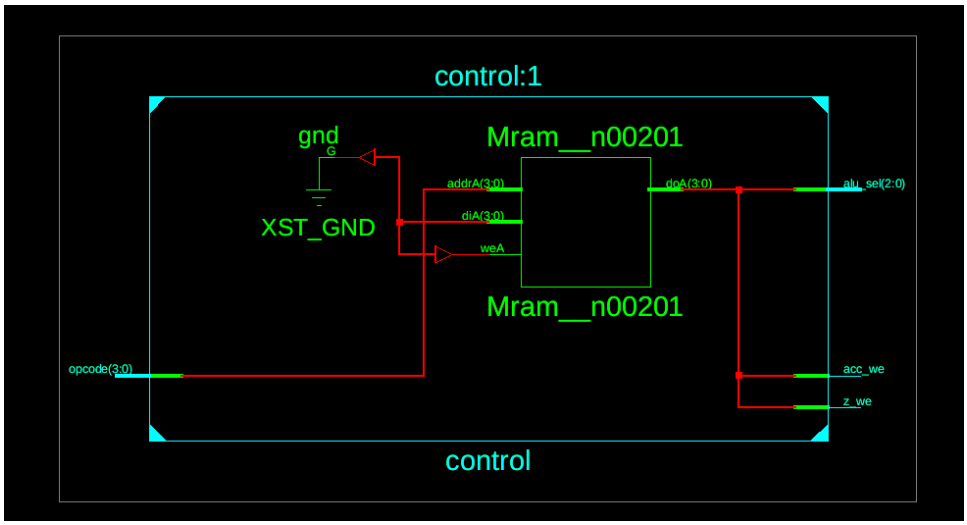


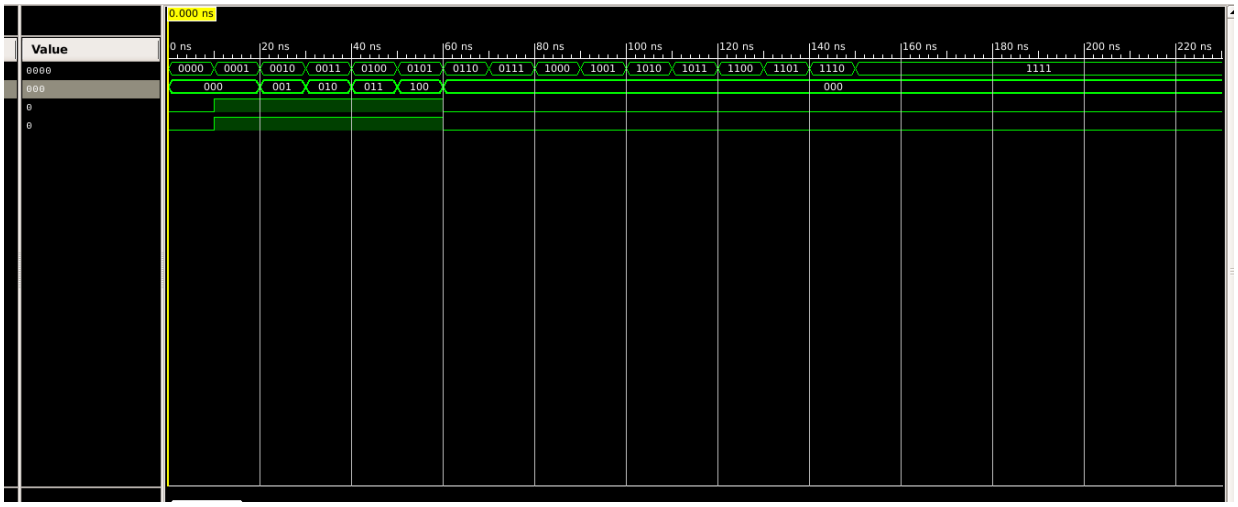
Figure 6: Expanded view of RTL schematic of the control unit (control)

The decoder was implemented by the synthesis tool as a small look-up memory (Mram__n00201) with the 4-bit opcode as address and a 4-bit output; three bits form alu_sel(2:0) and the remaining bit drives both acc_we and z_we.

Source code (control_tb.vhd):

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.ALL;
entity control_tb is end control_tb;
architecture sim of control_tb is
  signal opcode  : std_logic_vector(3 downto 0) := (others => '0');
  signal alu_sel : std_logic_vector(2 downto 0);
  signal acc_we, z_we : std_logic;
begin
  dut : entity work.control port map (opcode, alu_sel, acc_we, z_we);
  process
    variable errors : integer := 0;
    procedure chk(op : integer; sel : std_logic_vector(2 downto 0); en : std_logic) is
    begin
      opcode <= std_logic_vector(to_unsigned(op, 4)); wait for 10 ns;
      if acc_we /= en or z_we /= en or (en = '1' and alu_sel /= sel) then
        report "Control mismatch for opcode " & integer'image(op) severity error;
        errors := errors + 1;
      end if;
    end procedure;
  begin
    chk(0, "000", '0');   -- NOP
    chk(1, "000", '1');   -- LDI
    chk(2, "001", '1');   -- ADD
    chk(3, "010", '1');   -- SUB
    chk(4, "011", '1');   -- AND
    chk(5, "100", '1');   -- OR
    for op in 6 to 15 loop chk(op, "000", '0'); end loop;  -- unused -> NOP
    if errors = 0 then report "CONTROL_TB: ALL TESTS PASSED"; end if;
    wait;
  end process;
end sim;
```

Waveform:



The opcode is stepped from 0000 to 1111 every 10 ns. alu_sel follows 000 (LDI), 001 (ADD), 010 (SUB), 011 (AND) and 100 (OR) for opcodes 0001 to 0101, and the two write enables are high only during these five opcodes (10 ns to 60 ns). For opcode 0000 and for all unused opcodes 0110 to 1111 the enables are 0 (NOP).

4.4 ALU (alu.vhd)

A combinational ALU that operates on the accumulator value a (ACC) and the 8-bit operand b. The 3-bit select alu_sel chooses between passing the operand through (LDI), addition (ADD), subtraction (SUB), bitwise AND and bitwise OR. ADD and SUB use unsigned arithmetic on 8-bit vectors (numeric_std), so the carry or borrow is discarded and the result wraps modulo 256.

alu_sel	Instruction	y (result)
000	LDI	b
001	ADD	a + b (carry discarded)
010	**SUB**	**a − b (modulo 256)**
011	**AND**	**a and b**
100	**OR**	**a or b**

Source code (alu.vhd):

```
-- Step 4: ALU. 8-bit combinational, works on ACC and the 8-bit immediate.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu is
  port ( a       : in  std_logic_vector(7 downto 0);  -- ACC
         b       : in  std_logic_vector(7 downto 0);  -- immediate
         alu_sel : in  std_logic_vector(2 downto 0);
         y       : out std_logic_vector(7 downto 0) );
end alu;

architecture rtl of alu is
begin
  process(a, b, alu_sel)
  begin
    case alu_sel is
      when "000"  => y <= b;                                          -- LDI
      when "001"  => y <= std_logic_vector(unsigned(a) + unsigned(b));    -- ADD
      when "010"  => y <= std_logic_vector(unsigned(a) - unsigned(b));    -- SUB (mod 256)
      when "011"  => y <= a and b;                                    -- AND
      when "100"  => y <= a or b;                                     -- OR
      when others => y <= a;
    end case;
  end process;
end rtl;
```

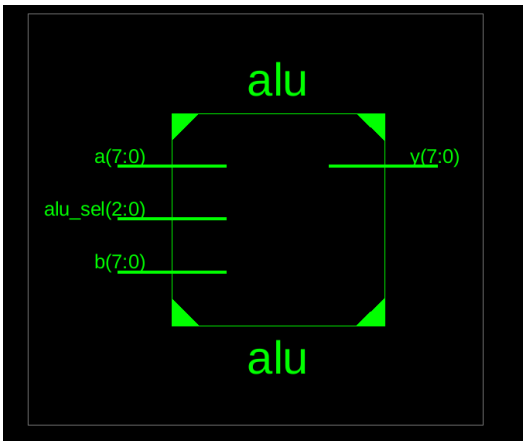


Figure 7: Top view of RTL schematic of the ALU (alu)

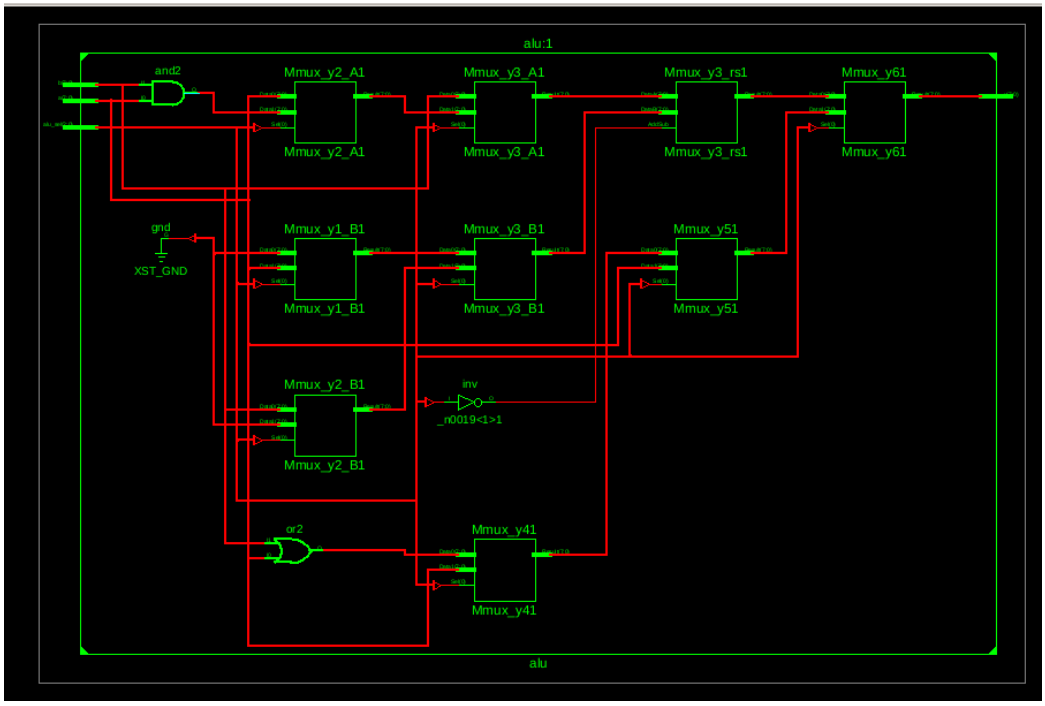


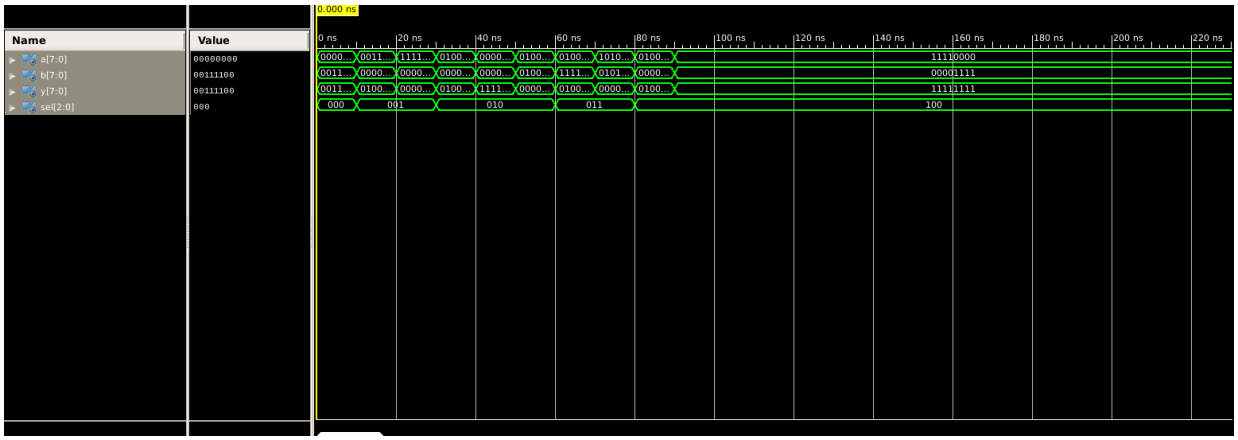
Figure 8: Expanded view of RTL schematic of the ALU (alu)

The synthesized ALU consists of an AND gate (and2), an OR gate (or2), an inverter and a tree of multiplexers (Mmux_…) controlled by alu_sel, which selects the result of the requested operation.

Source code (alu_tb.vhd):

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.ALL;
entity alu_tb is end alu_tb;
architecture sim of alu_tb is
  signal a, b, y : std_logic_vector(7 downto 0) := (others => '0');
  signal sel : std_logic_vector(2 downto 0) := "000";
begin
  dut : entity work.alu port map (a, b, sel, y);
  process
    variable errors : integer := 0;
    procedure chk(av, bv : integer; s : std_logic_vector(2 downto 0); exp : integer; name : string) is
    begin
      a <= std_logic_vector(to_unsigned(av, 8)); b <= std_logic_vector(to_unsigned(bv, 8));
      sel <= s; wait for 10 ns;
      if to_integer(unsigned(y)) /= exp then
        report name & " failed: a=" & integer'image(av) & " b=" & integer'image(bv) &
               " got " & integer'image(to_integer(unsigned(y))) & " expected " & integer'image(exp)
               severity error;
        errors := errors + 1;
      end if;
    end procedure;
  begin
    chk(16#00#, 16#3C#, "000", 16#3C#, "LDI");
    chk(16#3C#, 16#0F#, "001", 16#4B#, "ADD");
    chk(16#FF#, 16#01#, "001", 16#00#, "ADD wrap");
    chk(16#4B#, 16#0B#, "010", 16#40#, "SUB");
    chk(16#00#, 16#01#, "010", 16#FF#, "SUB wrap");
    chk(16#4A#, 16#4A#, "010", 16#00#, "SUB zero");
    chk(16#40#, 16#F0#, "011", 16#40#, "AND");
    chk(16#AA#, 16#55#, "011", 16#00#, "AND zero");
    chk(16#40#, 16#0A#, "100", 16#4A#, "OR");
    chk(16#F0#, 16#0F#, "100", 16#FF#, "OR full");
    if errors = 0 then report "ALU_TB: ALL TESTS PASSED"; end if;
    wait;
  end process;
end sim;
```

Waveform:



The testbench applies ten test vectors. sel steps through 000 (LDI), 001 (ADD), 010 (SUB), 011 (AND) and 100 (OR) and y always equals the expected result. The first vector (a = 00000000, b = 00111100, sel = 000) gives y = 00111100 (LDI 0x3C); the last vector (a = 11110000, b = 00001111, sel = 100) gives y = 11111111 (OR). The wrap-around cases ADD 0xFF + 0x01 = 0x00 and SUB 0x00 − 0x01 = 0xFF are also checked.

4.5 ACC and Z Registers (acc_z.vhd)

The sequential storage of the CPU: the 8-bit accumulator ACC (output q) and the 1-bit zero flag Z. Both are updated on the rising edge of clk and cleared by a synchronous, active-high reset (ACC = 0x00, Z = 1). When acc_we is asserted ACC loads the ALU result d; when z_we is asserted Z is set if d is 0x00 and cleared otherwise. When both enables are 0 (NOP) ACC and Z keep their values.

reset	acc_we / z_we	Action at the rising edge of clk
1	x	ACC ← 0x00, Z ← 1
0	acc_we = 1	ACC ← d
0	z_we = 1	Z ← 1 if d = 0x00, otherwise Z ← 0
0	0 / 0	ACC and Z unchanged (NOP)

Source code (acc_z.vhd):

```
-- Step 5: ACC (8-bit) and Z flag (1-bit). Sync active-high reset: ACC=0x00, Z=1.
-- (R register omitted: Set 1 has no STA/LDR.)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity acc_z is
  port ( clk, reset : in  std_logic;
         acc_we     : in  std_logic;
         z_we       : in  std_logic;
         d          : in  std_logic_vector(7 downto 0);  -- ALU result
         q          : out std_logic_vector(7 downto 0);
         z          : out std_logic );
end acc_z;

architecture rtl of acc_z is
  signal acc_r : std_logic_vector(7 downto 0) := (others => '0');
  signal z_r   : std_logic := '1';
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if reset = '1' then
        acc_r <= (others => '0');
        z_r   <= '1';
      else
        if acc_we = '1' then acc_r <= d; end if;
        if z_we = '1' then
          if d = x"00" then z_r <= '1'; else z_r <= '0'; end if;
        end if;
      end if;
    end if;
  end process;
  q <= acc_r;
  z <= z_r;
end rtl;
```

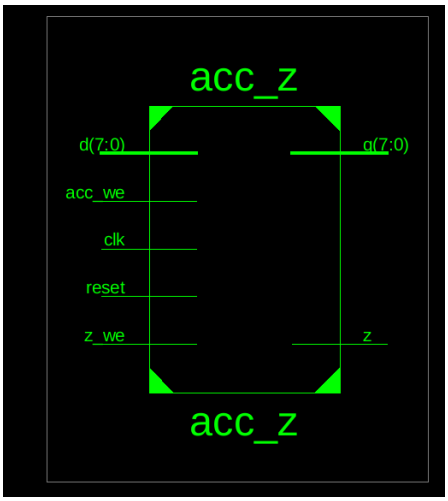


Figure 9: Top view of RTL schematic of the ACC and Z registers (acc_z)

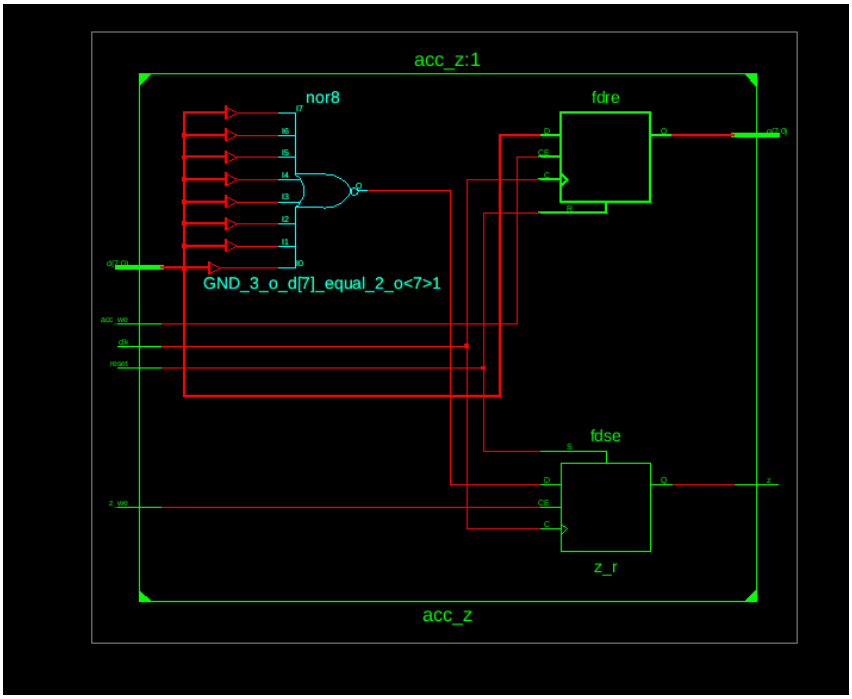


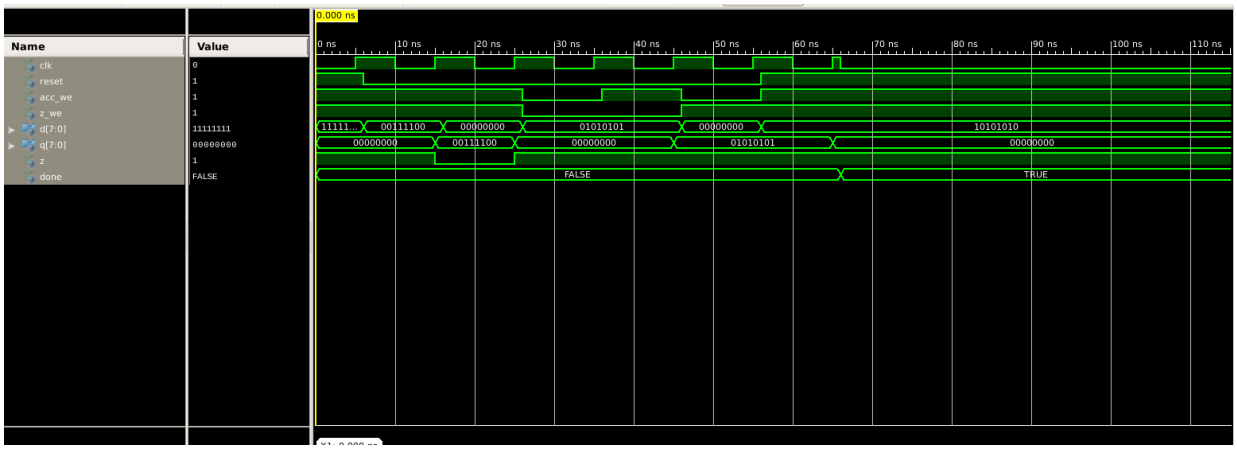
Figure 10: Expanded view of RTL schematic of the ACC and Z registers (acc_z)

ACC is implemented with eight FDRE flip-flops (clock enable = acc_we, synchronous reset = reset). The zero detection is an 8-input NOR gate on d, and the Z flag is an FDSE flip-flop (clock enable = z_we) whose synchronous set input provides the reset value 1.

Source code (acc_z_tb.vhd):

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
entity acc_z_tb is end acc_z_tb;
architecture sim of acc_z_tb is
  signal clk, reset, acc_we, z_we : std_logic := '0';
  signal d, q : std_logic_vector(7 downto 0) := (others => '0');
  signal z : std_logic;
  signal done : boolean := false;
begin
  dut : entity work.acc_z port map (clk, reset, acc_we, z_we, d, q, z);
  clk <= not clk after 5 ns when not done else '0';
  process
    variable errors : integer := 0;
    procedure step(dv : std_logic_vector(7 downto 0); aw, zw : std_logic;
                   expq : std_logic_vector(7 downto 0); expz : std_logic; name : string) is
    begin
      d <= dv; acc_we <= aw; z_we <= zw;
      wait until rising_edge(clk); wait for 1 ns;
      if q /= expq or z /= expz then
        report name & " failed" severity error; errors := errors + 1;
      end if;
    end procedure;
  begin
    reset <= '1';
    step(x"FF", '1', '1', x"00", '1', "Reset priority");
    reset <= '0';
    step(x"3C", '1', '1', x"3C", '0', "Load 3C");
    step(x"00", '1', '1', x"00", '1', "Load 00 sets Z");
    step(x"55", '0', '0', x"00", '1', "NOP holds");
    step(x"55", '1', '0', x"55", '1', "ACC only, Z unchanged");
    step(x"00", '0', '1', x"55", '1', "Z only");
    reset <= '1';
    step(x"AA", '1', '1', x"00", '1', "Reset again");
    if errors = 0 then report "ACC_Z_TB: ALL TESTS PASSED"; end if;
    done <= true; wait;
  end process;
end sim;
```

Waveform:



The waveform shows the sequence of the testbench: reset forces q = 00000000 and z = 1; loading 0x3C gives q = 00111100 and z = 0; loading 0x00 clears q and sets z = 1 again; with both enables 0 the register keeps its value although d = 0x55; with only acc_we = 1, q = 01010101 but z stays 1; with only z_we = 1, q is unchanged; finally reset clears q to 0x00 again. done becomes TRUE at the end and stops the clock.

5. Final CPU Integration and Verification

5.1 Top-level module (cpu.vhd)

The top-level module connects the program counter, the program ROM, the control unit, the ALU and the ACC/Z registers. The program counter addresses the ROM; the upper four bits of the instruction go to the control unit and the lower eight bits go to the ALU as the operand; the ALU result and the control signals are applied to the ACC/Z registers, which are updated at the rising edge of the clock. The top-level block exposes clk and reset as inputs and q[7:0], z and pc_o[3:0] as outputs.

Source code (cpu.vhd):

```
-- Step 6: Top level. Connects PC, ROM, Control, ALU, ACC/Z.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cpu is
  port ( clk, reset : in  std_logic;
         q          : out std_logic_vector(7 downto 0);  -- ACC
         z          : out std_logic;
         pc_o       : out std_logic_vector(3 downto 0) );
end cpu;

architecture structural of cpu is
  signal pc_s    : std_logic_vector(3 downto 0);
  signal instr   : std_logic_vector(11 downto 0);
  signal alu_sel : std_logic_vector(2 downto 0);
  signal acc_we, z_we : std_logic;
  signal acc_s, alu_y : std_logic_vector(7 downto 0);
begin
  u_pc  : entity work.pc     port map (clk, reset, pc_s);
  u_rom : entity work.rom    port map (pc_s, instr);
  u_ctl : entity work.control port map (instr(11 downto 8), alu_sel, acc_we, z_we);
  u_alu : entity work.alu    port map (acc_s, instr(7 downto 0), alu_sel, alu_y);
  u_reg : entity work.acc_z  port map (clk, reset, acc_we, z_we, alu_y, acc_s, z);
  q    <= acc_s;
  pc_o <= pc_s;
end structural;
```

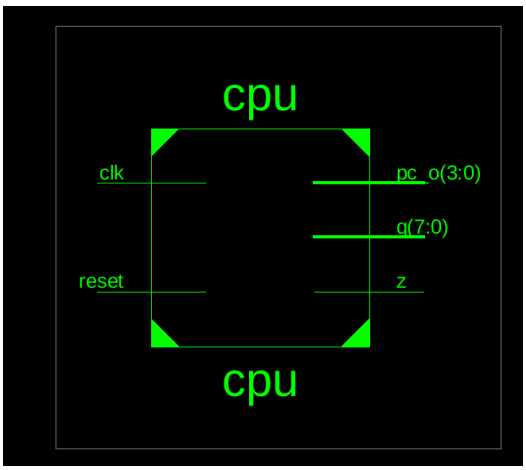


Figure 11: Top-level RTL schematic of the CPU (cpu)

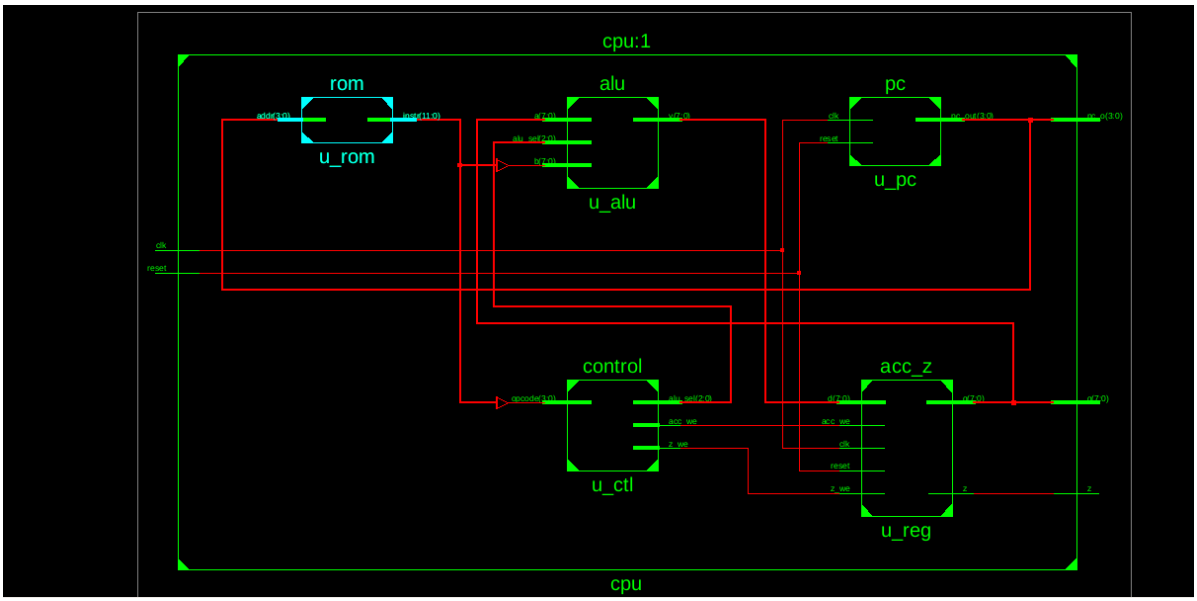


Figure 12: Internal RTL schematic of the CPU showing u_pc, u_rom, u_ctl, u_alu and u_reg

5.2 Final testbench (cpu_tb.vhd)

The testbench generates a 10 ns clock (rising edges at 5, 15, 25 ns and so on), holds reset high for one rising edge, releases it, and then lets the CPU execute the seven instructions of the Set 1 program, one per clock cycle. One nanosecond after each rising edge the outputs q, z and pc_o are compared with the expected values of Appendix B, which are stored in constant arrays; the values are also printed with a report statement. A mismatch is reported by an assert/report statement with the cycle number. At the end the testbench prints a completion message and stops the clock with the done flag.

Source code (cpu_tb.vhd):

```
-- Step 7: Testbench. Clock, RESET for one rising edge, then run 7 instructions
-- and check ACC / Z / next-PC against Appendix B (Set 1).
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity cpu_tb is end cpu_tb;

architecture sim of cpu_tb is
  signal clk, reset, z : std_logic := '0';
  signal q    : std_logic_vector(7 downto 0);
  signal pc_o : std_logic_vector(3 downto 0);
  signal done : boolean := false;

  -- VHDL-93 replacement for to_hstring (ISE does not support VHDL-2008)
  function to_hex(v : std_logic_vector) return string is
    constant HEX : string(1 to 16) := "0123456789ABCDEF";
    variable n   : integer := (v'length + 3) / 4;
    variable pad : std_logic_vector(n*4-1 downto 0) := (others => '0');
    variable res : string(1 to n);
    variable nib : integer;
  begin
    pad(v'length-1 downto 0) := v;
    for i in 1 to n loop
      nib := 0;
      for j in 3 downto 0 loop
        nib := nib * 2;
        if pad((n-i)*4 + j) = '1' then nib := nib + 1; end if;
      end loop;
      res(i) := HEX(nib + 1);
    end loop;
    return res;
  end function;

  type exp_acc_t is array (0 to 7) of std_logic_vector(7 downto 0);
  type exp_z_t   is array (0 to 7) of std_logic;
  type exp_pc_t  is array (0 to 7) of std_logic_vector(3 downto 0);
  constant EXP_ACC : exp_acc_t := (x"00",x"3C",x"4B",x"40",x"40",x"4A",x"00",x"FF");
  constant EXP_Z   : exp_z_t   := ('1','0','0','0','0','0','1','0');
  constant EXP_PC  : exp_pc_t  := (x"0",x"1",x"2",x"3",x"4",x"5",x"6",x"7");
begin
  dut : entity work.cpu port map (clk, reset, q, z, pc_o);

  clk <= not clk after 5 ns when not done else '0';

  stim : process
    variable errors : integer := 0;
  begin
    reset <= '1';
    wait until rising_edge(clk);          -- cycle 0: RESET edge
    wait for 1 ns; reset <= '0';
    for i in 0 to 7 loop
      if i > 0 then wait until rising_edge(clk); wait for 1 ns; end if;
      report "Cycle " & integer'image(i) & "  ACC=" & to_hex(q) &
             "  Z=" & std_logic'image(z) & "  nextPC=" & to_hex(pc_o);
      if q /= EXP_ACC(i) or z /= EXP_Z(i) or pc_o /= EXP_PC(i) then
        report "MISMATCH at cycle " & integer'image(i) severity error;
        errors := errors + 1;
      end if;
    end loop;
    if errors = 0 then report "ALL TESTS PASSED" severity note;
    else report integer'image(errors) & " MISMATCHES" severity failure; end if;
    done <= true;
    wait;
  end process;
end sim;
```

5.3 Simulation results

Expected results from Appendix B for Set 1 (values after each rising clock edge):

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC	Clock edge
0	-	RESET = 1	0x00	1	00000000	0	5 ns
1	0	LDI 0x3C	0x3C	0	00111100	1	15 ns
2	1	ADD 0x0F	0x4B	0	01001011	2	25 ns
3	**2**	**SUB 0x0B**	**0x40**	**0**	**01000000**	**3**	**35 ns**
4	**3**	**AND 0xF0**	**0x40**	**0**	**01000000**	**4**	**45 ns**
5	**4**	**OR 0x0A**	**0x4A**	**0**	**01001010**	**5**	**55 ns**
6	**5**	**SUB 0x4A**	**0x00**	**1**	**00000000**	**6**	**65 ns**
7	**6**	**SUB 0x01**	**0xFF**	**0**	**11111111**	**7**	**75 ns**

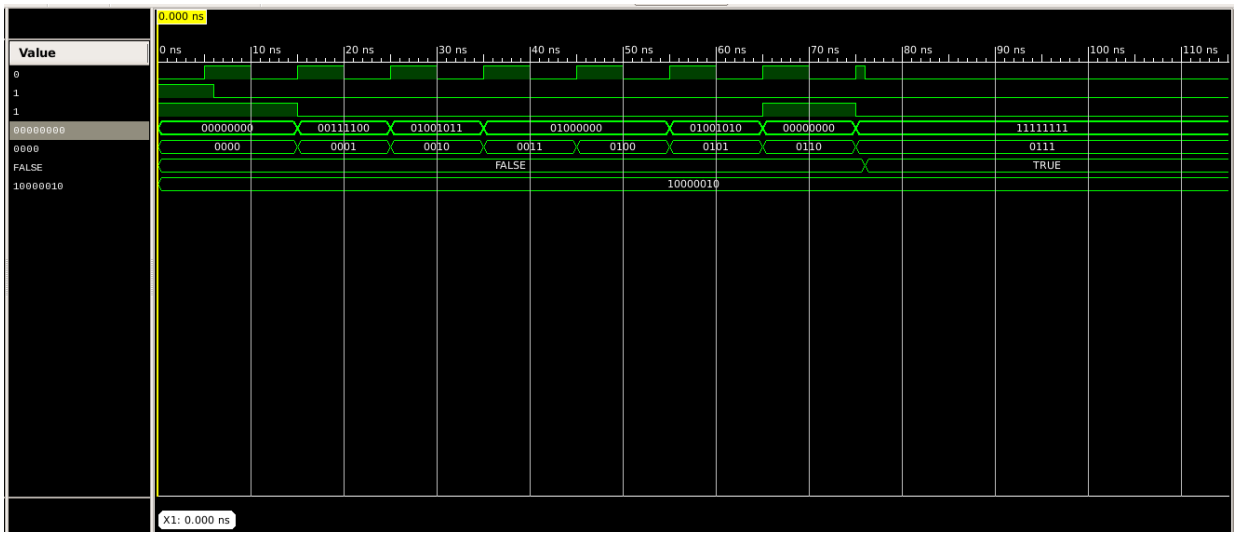


Figure 13: Simulation waveform of cpu from cpu_tb.vhd (clk, reset, z, q, pc_o and done)

The waveform agrees with the expected results of the table above:

- **RESET.** reset is 1 only at the very beginning and is released right after the first rising edge, so ACC = 0x00, Z = 1 and PC = 0 at the start.
- **ACC (q).** From the second rising edge onwards q changes once per clock cycle for the seven instructions (0x3C, 0x4B, 0x40, 0x40, 0x4A, 0x00, 0xFF). The value 0x40 stays for two cycles because AND 0xF0 applied to 0x40 gives 0x40 again; the last value 0xFF is the result of SUB 0x01 on 0x00 (wrap-around modulo 256).
- **Z flag (z).** z is 1 after reset and falls to 0 when LDI 0x3C is executed. It rises to 1 for exactly one clock cycle after SUB 0x4A (cycle 6), because 0x4A − 0x4A = 0x00, and returns to 0 after SUB 0x01. For all other instructions the result is non-zero, so Z stays 0.
- **Program counter (pc_o).** pc_o counts 0, 1, 2, …, 7 – one step per instruction. Its final value 0111 (= 7) is the address of the next instruction; it is a NOP (0x000) and the testbench stops the clock after the last check.
- **End of simulation.** done changes from FALSE to TRUE at about 76 ns, right after the last check, and stops the clock, so all signals stay constant afterwards.

The simulated ACC, Z and PC values agree with the Appendix B table in every cycle.

6. Version Control and Repository

The complete VHDL source code and the testbenches of the project are listed below.

File	Description
pc.vhd	4-bit program counter
rom.vhd	Program ROM (Set 1 program)
control.vhd	Control unit (opcode decoder)
alu.vhd	ALU (LDI, ADD, SUB, AND, OR)
acc_z.vhd	ACC register and Z flag
cpu.vhd	Top-level integration of all modules
pc_tb.vhd, rom_tb.vhd, control_tb.vhd, alu_tb.vhd, acc_z_tb.vhd	Self-checking testbenches of the individual modules
cpu_tb.vhd	Self-checking testbench for the Set 1 program

GitHub: https://github.com/Nazmul-SWE/Mini-8bit-accumulator-cpu

7. Challenges Faced During the Project

- Understanding how a 12-bit instruction is divided into a 4-bit opcode and an 8-bit operand, and encoding the Set 1 program correctly as machine code in the ROM (for example SUB 0x4A = 0x34A).
- Implementing SUB and ADD so that the result wraps modulo 256 and the carry or borrow is discarded (for example 0x00 − 0x01 = 0xFF), which was done with unsigned arithmetic on 8-bit vectors.
- Making Z follow ACC correctly: Z must change only when a value is written to ACC, so a NOP or an unused opcode must leave both ACC and Z unchanged. This is solved by the write enables acc_we and z_we and by giving every control output a default NOP value.
- Getting the reset behaviour right: a synchronous, active-high reset applied for exactly one rising edge that sets ACC = 0, Z = 1 and PC = 0.
- Tool compatibility: Xilinx ISE supports VHDL-93 only, so the function to_hstring (VHDL-2008) used to print hexadecimal values in the first version of the testbench was reported as “not declared”. It was replaced by a small user-defined function to_hex inside the testbench.
- Making the testbenches self-checking: the expected values of Appendix B are stored in constant arrays and compared with assert statements one nanosecond after every rising edge, so each cycle is checked automatically instead of by eye.

8. Conclusion

A mini 8-bit accumulator CPU supporting the Set 1 instructions (LDI, ADD, SUB, AND and OR) was designed in VHDL and verified in simulation using Xilinx ISE. The design is modular, consisting of a program counter, a program ROM, a control unit, an ALU and the ACC/Z registers, connected by a top-level module. All sequential blocks operate on the rising edge of the clock with a synchronous, active-high reset. Every module was tested with its own self-checking testbench, and the simulation of the complete Appendix B program agrees with the expected ACC, Z and PC values in every cycle, including the one-cycle Z flag after SUB 0x4A and the modulo-256 wrap-around of the last SUB 0x01. The project demonstrates how a few small, well-defined modules can be combined into a complete datapath with a fetch, decode and execute cycle.